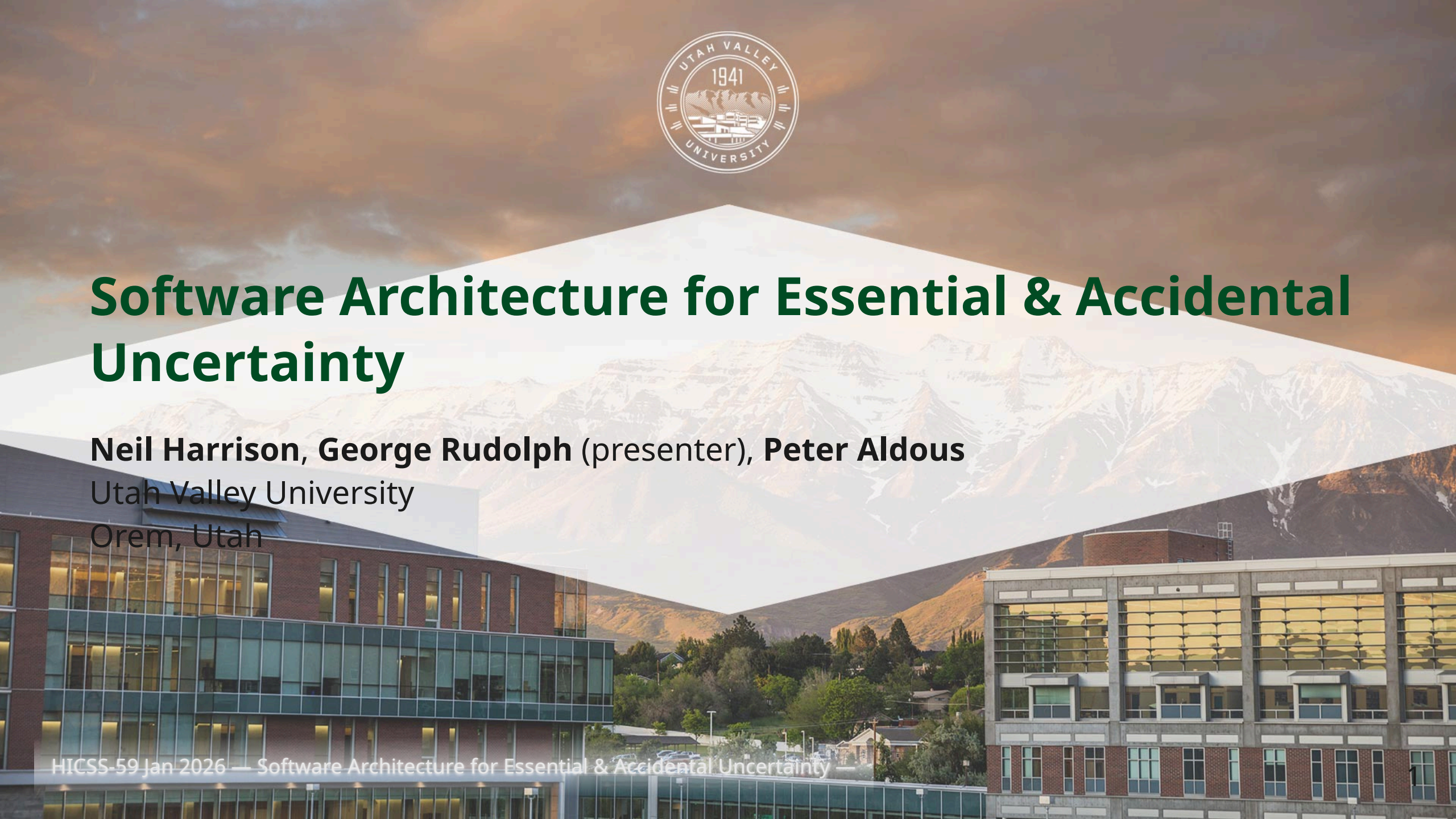


Software Architecture for Essential & Accidental Uncertainty

Neil Harrison, **George Rudolph** (presenter), **Peter Aldous**
Utah Valley University
Orem, Utah

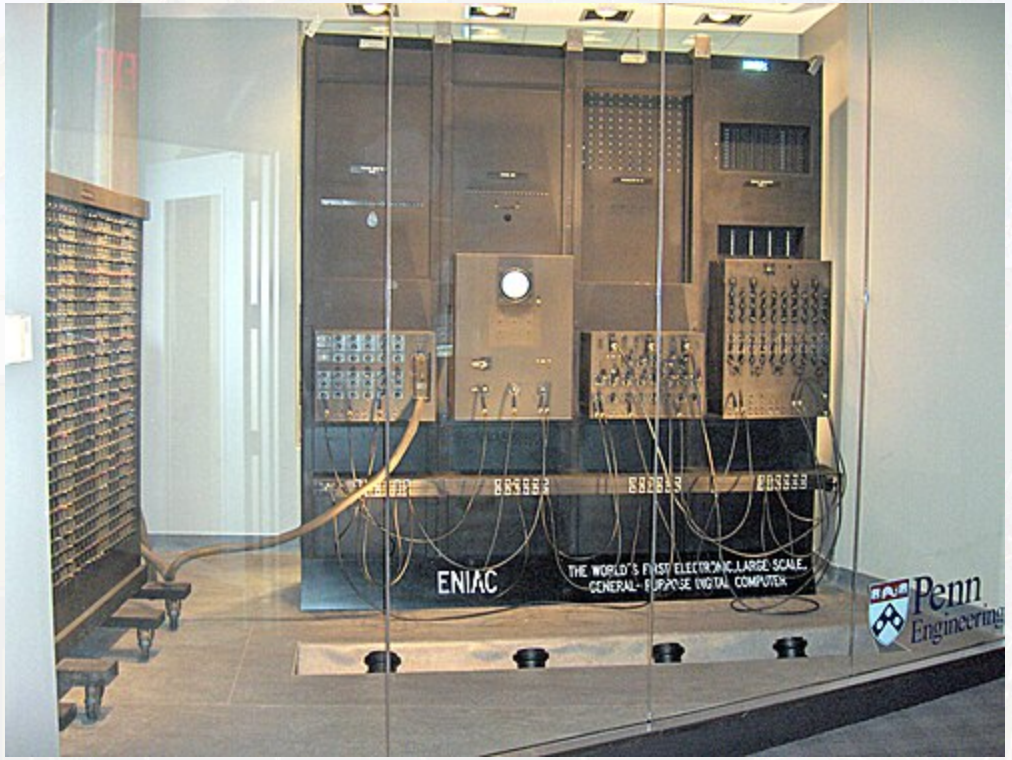
The Challenge: From Deterministic to Uncertain

Early computing (ENIAC, 1945-1955): Precisely specified problems with deterministic outputs

Today: Computing expanded to business, communication, control systems, and AI

- Inputs, outputs, and processes became more varied and less precise
- Modeling non-mathematical problems via mathematical abstractions

We must now deal with uncertainty — and understand how to approach it in architecture and design.



Current State of the Art

- (2010) **David Garlan** called for uncertainty to be a first-class concern in software design and development.
- Current research and practice focus on **fault tolerance** and related tactics.
- **This misses a large part of uncertainty** and motivates a deeper investigation.

Essence vs. Accident (Brooks, 1986)

Accidental complexity: created by engineering choices, mitigated by better tools and abstractions.
Essential complexity: inherent in the problem — cannot be removed.

Likewise, there is essential and accidental uncertainty.

Essential and Accidental Uncertainty

- **Accidental uncertainty**: can be separated from the problem; *a transformation exists* to remove it.
- **Essential uncertainty**: must be addressed as part of the problem; *no transformation* can remove it.

Computing is the processing of data. Consider uncertainty across:

- **Input** (can be essential or accidental)
- **Transformation** from input to output (can be essential or accidental)
- **Output** (often essential)

Uncertainty in the Transformation (Algorithms)

The **algorithm or processing logic** that transforms input to output can *introduce* uncertainty

- **Accidental** → can be fixed
  - Hardware glitches
  - Compiler differences
  - Implementation bugs
- **Essential**
  - No deterministic algorithm exists
  - Example: Image recognition

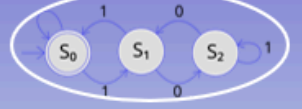
- **Essential (intractable)**
  - Algorithm is computationally infeasible
  - Approximations introduce uncertainty
  - Examples: Game of Go, optimization problems

Critical insight: Essential transformation uncertainty often reflects **uncertain output** — requires probabilistic or AI-based approaches

Essential Output Uncertainties

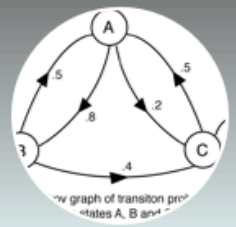
Output characteristics	Example
There is a known right answer	Mathematical computation
The right answer(s) can be known	Sales website (How well did a product sell?)
Right answer exists but is infeasible to compute directly	Game of Go (astronomical state space)
Unknown right answer; we can **guess**	Image recognition
Effect is unknown, but can be measured	Optimization problems
Effect is unknown; system is **chaotic**	Weather forecasting
No specific answer, but can observe "right" direction	Advertising; many simulation problems
No single right answer; the problem is **vague**	Automated prose writing
The problem itself is unknown	Grounded theory

Spectrum of Uncertainty



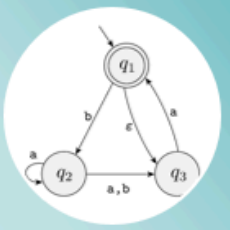
Deterministic

- Exactly one action possible for an input from a given program state
- Always the same action for input, state pairs
- No uncertainty



Probabilistic

- More than one action possible for an input from a given program state
- Each transition has a known probability
- Probabilities out of each possible state add to 1
- Some uncertainty



Nondeterministic

- More than one action possible for an input from a given program state
- We don't know the transition probabilities
- And/or cannot easily determine possible probabilities
- High uncertainty

Less Uncertainty ← → More Uncertainty

Architectural Approaches

Different levels and characteristics of uncertainty suggest different architectural styles, patterns, and tactics

- **Virtually no uncertainty** → Batch processing
- **Accidental uncertainty** → Pipes & Filters; Layers; Broker
- **Greater essential uncertainty** → AI-based approaches
  - Blackboard
  - Neural networks
  - Machine learning / deep learning

Spectrum of Architectural Approaches

An Architecture Process for Uncertainty

1. **Identify** uncertainties associated with the system.
2. For each, determine if it is **essential** or **accidental**.
3. **Study the source** of uncertainty (external is often accidental).
4. Determine if a **transformation** can remove the uncertainty.
5. If so, apply **fault-tolerance** and related techniques. If not, use AI-based approaches.

Key questions:

- **Input**: Is it due to imprecise specification or data variability?
- **Output**: Is there a known correct output? Can it be deterministically derived?
- **Transformation**: Is the relationship deterministic? If so, is it tractable?

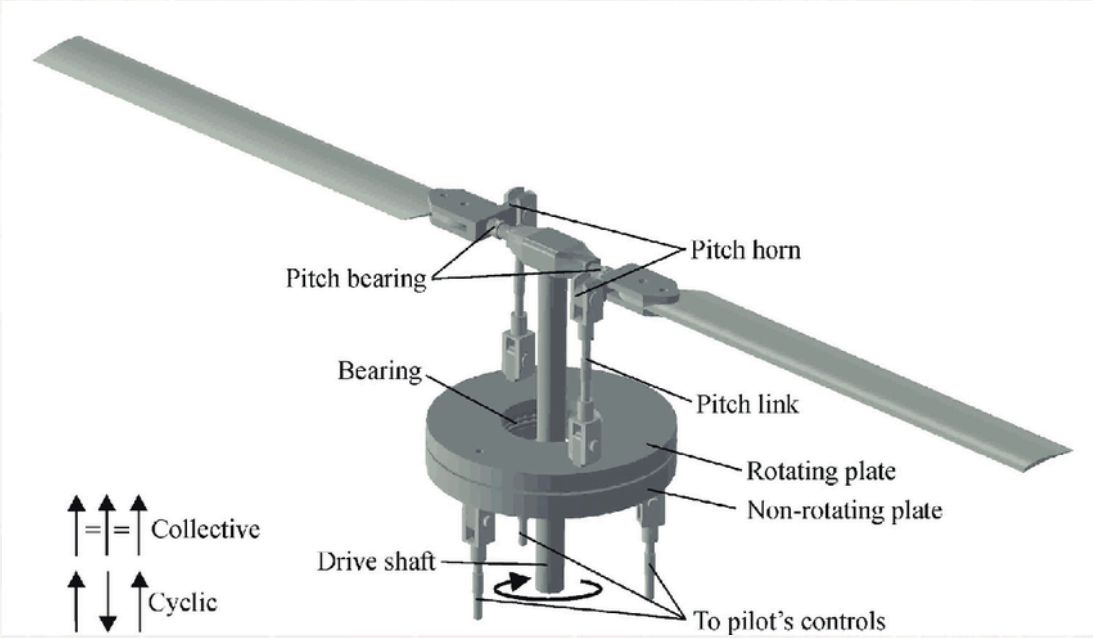
Case Study: Helicopter Rotor Balancing

Problem: Unbalanced rotor causes vibration, wear, loss of power, safety issues

Uncertainty Analysis:

- **Input**: Known measurements; unique wear patterns (essential)
- **Output**: Measurable (IPS); multiple solution sequences possible
- **Processing**: Initially appeared non-deterministic/chaotic

Initial approach: Neural network (treating as essential uncertainty)



Case Study: Helicopter Rotor Balancing (continued)

Further analysis revealed:

- Each helicopter has a **unique wear profile**
- Initial NN trained on many helicopters → generic approximation
- **Adjustment/response history** reveals unique profiles
- **Key insight**: There exists a **deterministic, repeatable** sequence of adjustments with **feedback loops**

What appeared essential was actually accidental, solvable with the right approach.

Results: System balances rotors to **0.05 IPS** — *10× better* than the 0.5 IPS standard. Actively marketed to US Army.

Case Study: Online Bookstore Evolution

Initially: Inputs/outputs well-specified; numerous **accidental uncertainties** (e.g., unreliable networks). Straightforward architecture.

Evolution 1 — Related Books: Which books are related? An **essential uncertainty** — output cannot be known beforehand, no single right answer. Approaches become **probabilistic** (recommendation algorithms).

Evolution 2 — Promotions: Using browsing history introduces even more uncertainty. Requires **machine learning** approaches, mapping to different positions on the uncertainty spectrum.

References

- Harrison, N.B., Rudolph, G., and Aldous, P. *Software Architecture for Essential and Accidental Uncertainty*. HICSS, 2026.
- Harrison, N.B., Rudolph, G., and Aldous, P. *Is Artificial Intelligence the Answer to Everything?* Intermountain Conference on Engineering, Technology and Computing (IETC), 2025.

Thank you!

Summary: We've presented a framework for understanding essential versus accidental uncertainty in software architecture. This distinction is crucial for selecting appropriate architectural approaches. Essential uncertainty requires probabilistic and AI-based methods, while accidental uncertainty can be addressed with traditional fault-tolerance techniques.

Questions?